Gérer des bases de données MySQL avec MySQLi dans PHP

Table des matières

Introduction : 1
MySQLi (Orienté Objet) : 3
Connexion au serveur 3
Fermer la connexion : 3
Créer une base de données 3
Créer une table 4
Insérer des données dans la table 6
Obtenir l'ID du dernier enregistrement inséré 7
Insérer plusieurs enregistrements 8
Instructions préparées 9
Insérer des données à partir d'une variable 11
Insérer des données avec le nom des champs dans des variables 12
Sélectionner des données 12
Supprimer des données de MySQL 13
Mettre à jour les données 14
Limiter les sélections de données 15

Introduction :

Les dernières versions de PHP (à partir de la 5) peuvent fonctionner avec une base de données MySQL en utilisant:

- Extension MySQLi (le "i" signifie "improved" amélioré)
- PDO (PHP Data Objects)

Les versions précédentes de PHP utilisaient l'extension MySQL. Cependant, cette extension est devenue obsolète en 2012.

MySQLi ou PDO avantages-inconvénients :

PDO fonctionne sur différents systèmes de base de données, alors que MySQLi ne fonctionne qu'avec des bases de données MySQL.

Ainsi, si vous devez utiliser une autre base de données dans votre projet, PDO simplifie le processus. Il vous suffit de modifier la chaîne de connexion et quelques requêtes. Avec MySQLi, vous devrez réécrire tout le code - requêtes incluses.

Les deux sont orientés objet, mais MySQLi offre également une API procédurale.

Les deux supportent les déclarations préparées. Les instructions préparées protègent de l'injection SQL et sont très importantes pour la sécurité des applications Web.

Il y a bien trois méthodes :

MySQLi (orienté objet)
MySQLi (procédural)
PDO

MySQLi (Orienté Objet) :

Connexion au serveur

Avant de pouvoir accéder aux données de la base de données MySQL, nous devons pouvoir nous connecter au serveur :

```
<?php
$servername = "localhost";
$username = "username";
$password = "password";

// Create connection - Créer connexion
$conn = new mysqli($servername, $username, $password);

// Check connection - Vérifier connexion
if ($conn->connect_error) {
    die("La connexion a échouée : " . $conn->connect_error);
}
echo "Connecter avec succès";
?>
```

Attention **connect_error** a été cassé dans les versions PHP 5.2.9 et 5.3.0. Si vous devez assurer la compatibilité avec les versions de PHP 5.2.9 et 5.3.0, utilisez plutôt le code suivant :

```
// Vérifier connexion
if (mysqli_connect_error ()) {
 die ("La connexion à la base de données a échouée:". mysqli_connect_error());
}
```

Fermer la connexion :

Il faut fermer la connexion pour cela :

```
$conn->close();
```

Créer une base de données

L'instruction CREATE DATABASE est utilisée pour créer une base de données dans MySQL.

L’exemple suivant crée une base de données nommée "myDB":

```
<?php
$servername = "localhost";
$username = "username";
$password = "password";
```

```
// Create connection
$conn = new mysqli($servername, $username, $password);
// Check connection
if ($conn->connect_error) {
    die("La connexion a échouée : " . $conn->connect_error);
}

// Create database
$sql = "CREATE DATABASE myDB";
if ($conn->query($sql) === TRUE) {
    echo "Database created successfully";
} else {
    echo "Error creating database: " . $conn->error;
}

$conn->close();
?>
```

Remarques :

- Lorsque vous créez une nouvelle base de données, vous ne devez spécifier que les trois premiers arguments de l'objet mysqli (nom_serveur, nom d'utilisateur et mot de passe).
- Le nom de la base de donnée est forcé en minuscule (il ne respecte pas la case).
- Si vous devez utiliser un port spécifique, ajoutez une chaîne vide pour l'argument *nom-base de données*, comme ceci :

```
$conn = new mysqli($servername, $username, $password, "", $port);
```

Créer une table

L'instruction CREATE TABLE permet de créer une table dans MySQL.

Nous allons créer une table nommée "Utilisateurs", avec cinq colonnes :

id	prenom	nom	email	reg_date
Clé primaire **Auto incrémentée**	VARCHAR(30)	VARCHAR(30)	VARCHAR(50)	TIMESTAMP

<u>Requête MySQL :</u>

CREATE TABLE Utilisateurs

(
id INT(6) UNSIGNED AUTO_INCREMENT PRIMARY KEY,
prenom VARCHAR(30) NOT NULL,
nom VARCHAR(30) NOT NULL,
email VARCHAR(50),
reg_date TIMESTAMP
)

Le type de données spécifie le type de données que la colonne peut contenir. Après le type de données, vous pouvez spécifier d'autres attributs facultatifs pour chaque colonne :

NOT NULL - Chaque ligne doit contenir une valeur pour cette colonne. Les valeurs NULL ne sont pas autorisées.

Valeur DEFAULT - Définit une valeur par défaut qui est ajoutée lorsqu'aucune autre valeur n'est transmise.

UNSIGNED - Utilisé pour les types de nombre, limite les données stockées aux nombres positifs et à zéro.

AUTO INCREMENT - MySQL augmente automatiquement la valeur du champ de 1 chaque fois qu'un nouvel enregistrement est ajouté.

PRIMARY KEY - Utilisé pour identifier de manière unique les lignes d'une table.Ce paramètre est souvent un numéro d'identification et est souvent utilisée avec AUTO_INCREMENT.

Remarques :

- Il ne faut pas utiliser d'accent dans le nommage des colonnes (prenom).

- La valeur NULL désigne qu'il n'y a pas de valeur, par exemple :
  - ✓ Une colonne « second prénom » peut ne pas contenir de valeur, beaucoup de personnes ont qu'un seul prénom.
  - ✓ Un colonne « date de fin » peut ne pas être connue lors de la création d'une table.

- Chaque table doit avoir une colonne de clé primaire (dans ce cas : la colonne "id"). Sa valeur doit être unique pour chaque enregistrement de la table.

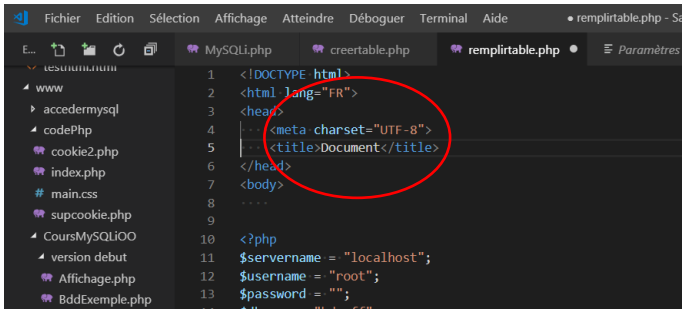
```
<?php
$servername = "localhost";
$username = "username";
$password = "password";
$dbname = "myDB";

// Create connection
$conn = new mysqli($servername, $username, $password, $dbname);
// Check connection
if ($conn->connect_error) {
    die("La connexion a échouée : " . $conn->connect_error);
}

// sql to create table
$sql = "CREATE TABLE Utilisateurs (
id INT(6) UNSIGNED AUTO_INCREMENT PRIMARY KEY,
prenom VARCHAR(30) NOT NULL,
nom VARCHAR(30) NOT NULL,
email VARCHAR(50),
reg_date TIMESTAMP
)";
```

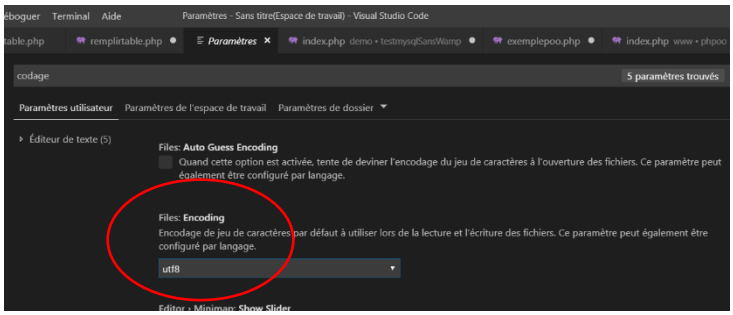
```
if ($conn->query($sql) === TRUE) {
    echo "Table Utilisateurs créée avec succès";
} else {
    echo "Erreur lors de la creation de la table: " . $conn->error;
}

$conn->close();
?>
```

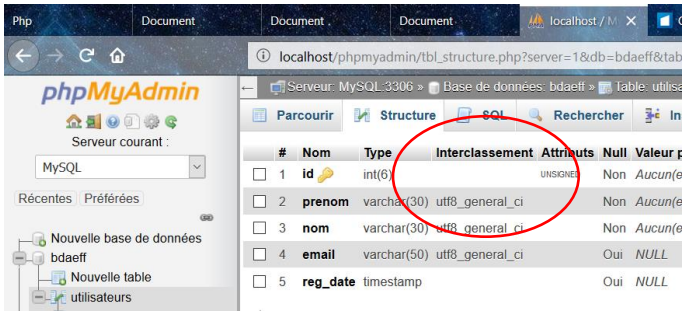
Insérer des données dans la table

Quelques règles de syntaxe à suivre :

- La requête SQL doit être entre quote en PHP.
- Les noms de colonne dans la requête SQL doivent être entre quote.
- Les valeurs des données (values) ne doivent pas être entre quote.
- Le mot NULL ne doit pas être entre quote.
- Les valeurs des données ne doivent pas contenir d'accent sauf si le codage utf8 est indiqué dans le code html, et dans la configuration de l'éditeur.







L'instruction INSERT INTO est utilisée pour ajouter de nouveaux enregistrements à une table MySQL :

```
INSERT INTO table_name ('column1', 'column2', 'column3',...)
VALUES (value1, value2, value3,...)
```

Remarque : Si une colonne est AUTO_INCREMENT (telle que la colonne "id") ou TIMESTAMP (telle que la colonne "reg_date"), il n'est pas nécessaire de la spécifier dans la requête SQL. MySQL ajoutera automatiquement la valeur.

L'exemple suivant ajoute un nouvel enregistrement à la table "Utilisateurs" créée précédemment :

```
<?php
$servername = "localhost";
$username = "username";
$password = "password";
$dbname = "myDB";

// Create connection
$conn = new mysqli($servername, $username, $password, $dbname);
// Check connection
if ($conn->connect_error) {
    die("La connexion a échouée : " . $conn->connect_error);
}

$sql = "INSERT INTO Utilisateurs (prenom, nom, email)
VALUES ('Jérôme', 'Burri', 'jerome.burri@gmail.com')";

if ($conn->query($sql) === TRUE) {
    echo "Nouveau enregistrement réussi";
} else {
    echo "Erreur: " . $sql . "<br>" . $conn->error;
}

$conn->close();
?>
```

Obtenir l'ID du dernier enregistrement inséré

Si nous effectuons un INSERT ou un UPDATE sur une table avec un champ AUTO_INCREMENT, nous pouvons obtenir immédiatement l'ID du dernier enregistrement inséré / mis à jour.

Dans la table "Utilisateurs", la colonne "id" est un champ AUTO_INCREMENT :

```
<?php
$servername = "localhost";
$username = "username";
$password = "password";
$dbname = "myDB";

// Create connection
$conn = new mysqli($servername, $username, $password, $dbname);
// Check connection
if ($conn->connect_error) {
    die("La connexion a échouée : " . $conn->connect_error);
}
```

```
$sql = "INSERT INTO Utilisateurs (prenom, nom, email)
VALUES ('Jérôme', 'Burri', 'jerome.burri@gmail.com')";

if ($conn->query($sql) === TRUE) {
    $last_id = $conn->insert_id;
echo " Nouveau enregistrement réussi. Le dernier id inséré est : " . $last_id;
}
else {
    echo "Erreur: " . $sql . "<br>" . $conn->error;
}

$conn->close();
?>
```

Insérer plusieurs enregistrements

Plusieurs instructions SQL doivent être exécutées avec la fonction *mysqli_multi_query ()*.

Les exemples suivants ajoutent trois nouveaux enregistrements à la table "Utilisateurs" :

```
// Create connection
$conn = new mysqli($servername, $username, $password, $dbname);
// Check connection
if ($conn->connect_error) {
    die("La connexion a échouée : " . $conn->connect_error);
}

$sql = "INSERT INTO Utilisateurs (prenom, nom, email)
VALUES ('Jérôme', 'Burri', 'jb@exemple.com');";
$sql .= "INSERT INTO Utilisateurs (prenom, nom, email)
VALUES ('Lydia', 'Burri', 'lb@exemple.com');";
$sql .= "INSERT INTO Utilisateurs (prenom, nom, email)
VALUES ('Bruce', 'Banner', 'bb@exemple.com')";

if ($conn->multi_query($sql) === TRUE) {
    echo " Nouveau enregistrement réussi. ";
} else {
    echo "Erreur: " . $sql . "<br>" . $conn->error;
}

$conn->close();
```

Instructions préparées

Une instruction préparée est une fonctionnalité utilisée pour exécuter les instructions SQL identiques (ou similaires) de manière répétée avec une grande efficacité.

Les instructions préparées fonctionnent essentiellement comme ceci :

Préparer :

Un modèle d'instruction SQL est créé et envoyé à la base de données.

Certaines valeurs sont laissées non spécifiées, elles sont appelées *paramètres* (étiquetés "?").

```
INSERT INTO Utilisateurs VALUES (?,?,?)
```

La base de données analyse, compile et optimise les requêtes sur le modèle d'instruction SQL et stocke le résultat sans l'exécuter.

Exécuter :

Ultérieurement, l'application lie les valeurs aux paramètres et la base de données exécute l'instruction. L'application peut exécuter l'instruction autant de fois qu'elle le souhaite avec des valeurs différentes.

Par rapport à l'exécution directe d'instructions SQL, les instructions préparées présentent trois avantages principaux :

- Les instructions préparées réduisent le temps d'analyse car la préparation de la requête est effectuée une seule fois (bien que l'instruction soit exécutée plusieurs fois).
- Les paramètres liés minimisent la bande passante sur le serveur, car vous ne devez envoyer que les paramètres à chaque fois, et non la requête entière.
- Les instructions préparées sont très utiles contre les injections SQL, car les valeurs de paramètre, qui sont transmises ultérieurement à l'aide d'un protocole différent, n'ont pas besoin d'être correctement échappées. Si le modèle d'instruction d'origine n'est pas dérivé d'une entrée externe, l'injection SQL ne peut pas avoir lieu.

```php
<?php
$servername = "localhost";
$username = "username";
$password = "password";
$dbname = "myDB";

// Create connection
$conn = new mysqli($servername, $username, $password, $dbname);

// Check connection
if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}
```

```
// prepare and bind - preparer et lier
$stmt = $conn->prepare("INSERT INTO Utilisateurs (prenom, nom, email) VALUES
(?, ?, ?)");
$stmt->bind_param("sss", $prénom, $nom, $email);

// set parameters and execute
$prénom = "Jérôme";
$nom = "Burri";
$email = "jb@exemple.com";
$stmt->execute();

$prénom = "Lydia";
$nom = "Burri";
$email = "lb@exemple.com";
$stmt->execute();

$prénom = "Bruce";
$nom = "Banner";
$email = "bb@exemple.com";
$stmt->execute();

echo "Nouveaux enregistrements créés ";

$stmt->close();
$conn->close();
?>
```

Dans la requête suivante :

`"INSERT INTO Utilisateurs (prénom, nom, email) VALUES (?,?,?)"`

Ici, nous insérons des points d'interrogations qui permettent aux VALUES de prendre des valeurs spécifiées plus loin.

Ensuite,
`$stmt-> bind_param("sss", $prénom, $nom, $ email);`

Cette fonction lie les paramètres à la requête SQL et indique à la base de données quels sont les paramètres.

L'argument "sss" répertorie les types de données que sont les paramètres.

Le caractère s indique à mysql que le paramètre est une chaîne de caractère (string).

L'argument peut être l'un des quatre types suivants :

- i - Integer
- d - Double
- s - String
- b - Blob

Le **BLOB**, pour binary large object, est un type de donnée permettant le stockage de données binaires (le plus souvent des fichiers de type image, son ou video) dans le champ d'une table d'une base de données.

Nous devons avoir un argument pour chaque paramètre.

En indiquant à mysql le type de données à attendre, nous minimisons le risque d'injections SQL.

Remarque : Si nous voulons insérer des données provenant de sources externes (comme une entrée utilisateur), il est très important que les données soient désinfectées et validées.

Insérer des données à partir d'une variable

On reprend un exemple d'instruction INSERT INTO, ici la table utilisateurs se présente comme ci-dessous :

id	nom	mdp	secret

Pour une colonne est AUTO_INCREMENT (telle que la colonne "id") il n'est pas nécessaire de la spécifier dans la requête SQL. MySQL ajoutera automatiquement la valeur.

Les données nom, mdp, secret peuvent être envoyées par un formulaire ou un logiciel de gestion de requêtes (Insomnia, postMan).

```
//fichier conf avec $servername, $username etc
    require('conf.php');


    if (!isset($_POST['nom']) || !isset($_POST['mdp']) ||
!isset($_POST['secret'])) {
        echo 'Pas de nom ou pas de mdp ou pas de secret !';
    } else {

        // pour simplifier :
        $nom = $_POST['nom'];
        $mdp = $_POST['mdp'];
        $secret = $_POST['secret'];

        // Create connection
        $conn = new mysqli($servername, $username, $password, $dbname);

        // Check connection
        if ($conn->connect_error) {
            die("Connection failed: " . $conn->connect_error);
        }

        //requete
        //sous la forme:
        //INSERT INTO Utilisateurs (prenom, nom, email)
        //ici la table Utilisateurs : id, nom, mdp, secret
```

```
        $sql = "INSERT INTO Utilisateurs (nom, mdp, secret) VALUES ('$nom',
'$mdp', '$secret')";

        if ($conn->query($sql) === TRUE) {
            $last_id = $conn->insert_id;
            echo " Nouveau enregistrement réussi. Le dernier id inséré est : "
. $last_id;
        } else {
            echo "Erreur: " . $sql . "<br>" . $conn->error;
        }

        $conn->close();
    }

    ?>
</body>

</html>
```

Dans la requête, les variables contenant les valeurs sont entre simple ‘ :

```
 $sql = "INSERT INTO Utilisateurs (nom, mdp, secret) VALUES ('$nom', '$mdp',
'$secret')";
```

Insérer des données avec le nom des champs dans des variables

Cet exemple reprend les contraintes de l’exemple précédent !

```
$table = 'Utilisateurs';
        $champs1 = 'nom';
        //requête
        //sous la forme :
        //INSERT INTO Utilisateurs (prenom, nom, email)
        //ici la table Utilisateurs : id, nom, mdp, secret
        $sql = "INSERT INTO $table ($champs1, mdp, secret) VALUES ('$nom',
'$mdp', '$secret')";
```

Dans la requête, les variables contenant les valeurs sont entre simple ‘ , les variables contenant la table et les champs n’ont pas de ‘ !

```
 $sql = "INSERT INTO $table ($champs1, mdp, secret) VALUES ('$nom', '$mdp',
'$secret')";
```

Sélectionner des données

L'instruction `SELECT` permet de sélectionner des données dans une ou plusieurs tables : `SELECT nom_colonne FROM nom_table`

où nous pouvons utiliser le caractère * pour sélectionner TOUTES les colonnes d'un tableau: `SELECT * FROM nom_table`

```
<?php
$servername = "localhost";
$username = "username";
$password = "password";
$dbname = "myDB";

// Create connection
$conn = new mysqli($servername, $username, $password, $dbname);
// Check connection
if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}

$sql = "SELECT id, prenom, nom FROM Utilisateurs";
$result = $conn->query($sql);

if ($result->num_rows > 0)
{
    // Pour chaque ligne on écrit id : 1 - Nom : Jérôme Burri
    while($row = $result->fetch_assoc())
      {
        echo "id : " . $row["id"]. " - Nom : " . $row["prenom"]. " " .
$row["nom"]. "<br>";
      }
}
else
{
    echo "0 results";
}
$conn->close();
?>
```

Supprimer des données de MySQL

L'instruction DELETE est utilisée pour supprimer des enregistrements d'une table :

DELETE FROM `nom_table` WHERE `some_column = some_value`

La clause WHERE spécifie le ou les enregistrements à supprimer. Si vous omettez la clause WHERE, tous les enregistrements seront supprimés !

L'exemple suivant efface l'enregistrement (la ligne) dont Id est 12, dans la table « dbaeff » suivante :

12	Jerome	Burri	jb@exemple.com	*NULL*
13	Lydia	Burri	lb@exemple.com	*NULL*
14	Bruce	Banner	bb@exemple.com	*NULL*

```php
<?php
$servername = "localhost";
$username = "root";
$password = "";
$dbname = "bdaeff";

// Create connection
$conn = new mysqli($servername, $username, $password, $dbname);
// Check connection
if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}

// sql to delete a record
$sql = "DELETE FROM Utilisateurs WHERE id=12";

if ($conn->query($sql) === TRUE) {
    echo "Donnée effacée avec succès";
} else {
    echo "Erreur lors de l'effacement : " . $conn->error;
}

$conn->close();
?>
```

Remarque : lors de l'effacement d'une ligne avec un critère gérant une chaine de caractère comme ci-dessous :

```php
// sql to delete a record
$sql = "DELETE FROM Utilisateurs WHERE email='jerome.burri@gmail.com'";
```

Il faut que la chaine de caractère soit entre quote.

Remarque : L'effacement, d'une ligne qui n'existe pas, ne crée pas d'erreur.

Mettre à jour les données

L'instruction UPDATE est utilisée pour mettre à jour des enregistrements existants dans une table :

UPDATE `nom_table` SET `colonne1` = `valeur`, `colonne2` = `valeur2`, ... WHERE `column` = `value`

La clause WHERE spécifie le ou les enregistrements à mettre à jour. Si vous omettez la clause WHERE, tous les enregistrements seront mis à jour !

L'exemple suivant remplace dans la table ci-dessous l'adresse mail existante par « lydia.burri@bidule.fr » pour les lignes ayant Lydia comme prénom.

d	prenom	nom	email	reg_date
6	Jerome	Burri	jb@example.com	NULL
7	Lydia	Burri	lb@example.com	NULL
8	Bruce	Banner	bb@example.com	NULL
9	Jerome	Burri	jb@exemple.com	NULL
0	Lydia	Burri	lb@exemple.com	NULL
1	Bruce	Banner	bb@exemple.com	NULL
3	Lydia	Burri	lb@exemple.com	NULL
4	Bruce	Banner	bb@exemple.com	NULL

```php
<?php
$servername = "localhost";$username = "root";$password = "";$dbname = "bdaeff";

$conn = new mysqli($servername, $username, $password, $dbname);
if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}

$sql = "UPDATE Utilisateurs SET email='lydia.burri@bidule.fr' WHERE prenom='lydia'";

if ($conn->query($sql) === TRUE) {
    echo "mise à jour réussie";
} else {
    echo "erreur lors de la mise à jour : " . $conn->error;
}
$conn->close();
?>
```

id	prenom	nom	email	reg_date
6	Jerome	Burri	jb@example.com	NULL
7	Lydia	Burri	lydia.burri@bidule.fr	NULL
8	Bruce	Banner	bb@example.com	NULL
9	Jerome	Burri	jb@exemple.com	NULL
10	Lydia	Burri	lydia.burri@bidule.fr	NULL
11	Bruce	Banner	bb@exemple.com	NULL
13	Lydia	Burri	lydia.burri@bidule.fr	NULL
14	Bruce	Banner	bb@exemple.com	NULL

Limiter les sélections de données

MySQL fournit une clause LIMIT utilisée pour spécifier le nombre d'enregistrements à renvoyer.

La clause LIMIT facilite le codage des résultats de plusieurs pages ou de la pagination avec SQL. Elle est très utile pour les tables volumineuses. Le renvoi d'un grand nombre d'enregistrements peut avoir une incidence sur les performances.

Sélection de tous les enregistrements de 1 à 30 (inclus) dans une table.

$sql = "SELECT * FROM Orders LIMIT 30";

Lorsque la requête SQL ci-dessus est exécutée, les 30 premiers enregistrements seront renvoyés.

Sélection des enregistrements 16 - 25 (inclus).

Mysql fournit également un moyen de gérer cela : en utilisant OFFSET. La requête SQL ci-dessous indique "ne renvoyez que 10 enregistrements, commencez par l'enregistrement 16 (OFFSET 15)" :

$sql = "SELECT * FROM Orders LIMIT 10 OFFSET 15";

Syntaxe plus courte pour obtenir le même résultat :

$sql = "SELECT * FROM Orders LIMIT 15, 10";

Notez que les nombres sont inversés lorsque vous utilisez une virgule.